



Solving the Lorenz ODE System Using Optimal ANN Architectures

Final Year Design Project I ■ Department of CSE, United International University

Team Members

Ikramul Hasan Moral (0112230489)
Md. Abu Bakar (0112230200)
Samiur Rahman Omlan (0112230195)
Fariha Islam (0112230431)
Md. Touhidul Islam (0112230435)

Supervisor

Dr. Muhammad Nomani Kabir
Professor, Department of CSE

Date

June 9, 2026

Outline

1. Problem & Motivation
2. Objectives
3. Background: Lorenz-1960 & ANNs
4. Literature Review
5. Research Gap & Questions
6. Methodology
7. Progress: Validated Baseline
8. Project Plan & FYDP-II
9. Conclusion

FYDP-I deliverable: a **validated numerical baseline** and a **controlled experimental design** for the ANN architecture search carried out in FYDP-II.

Problem & Motivation

No closed form

Coupled nonlinear ODEs like Lorenz-1960 rarely admit a closed-form solution.

Classical solvers are costly

RK4 is accurate but mesh-dependent and must *re-solve* step-by-step for every query.

Neural methods lean on physics

Most ODE networks add physics-informed losses or operator learning.

Central question: can a **plain feedforward ANN** — trained only on numerical reference data, with no physics in the loss — learn the Lorenz-1960 solution, and *which architecture does it best?*

Objectives

Build a trusted reference

- ▶ Solve Lorenz-1960 with RK4 *and* SciPy
- ▶ Cross-validate the two solvers
- ▶ Use the verified trajectory as ground truth

Search the architecture space

- ▶ Vary depth, width, activation
- ▶ Keep the network plain (no physics aids)

Evaluate accuracy vs. cost

- ▶ MSE, MAE, max error
- ▶ Training and inference time
- ▶ Convergence stability

Benchmark against literature

- ▶ Compare with PINN / DeepONet results
- ▶ Position plain ANN against physics-informed methods

Background: The Lorenz-1960 System

Three coupled nonlinear ODEs (a simplified meteorological model). With $k = 2$, $l = 1$:

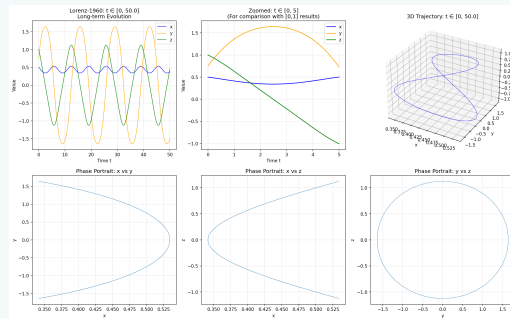
$$\dot{x} = kl \left(\frac{1}{k^2 + l^2} - \frac{1}{k^2} \right) yz$$

$$\dot{y} = kl \left(\frac{1}{l^2} - \frac{1}{k^2 + l^2} \right) xz$$

$$\dot{z} = \frac{kl^2}{2} \left(\frac{1}{k^2} - \frac{1}{l^2} \right) xy$$

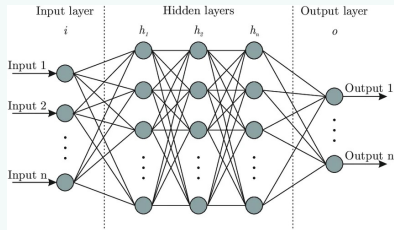
$x(0) = 0.5$, $y(0) = 0.75$, $z(0) = 1$, $t \in [0, 1]$.

Nonlinear and coupled — a compact but demanding benchmark for any solver.

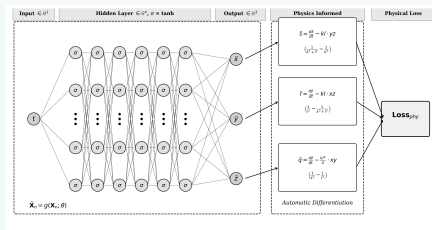


Long-term evolution, phase portraits, and 3D trajectory.

Background: Plain ANN vs. Physics-Informed NN



Plain ANN — data-driven, learns $t \mapsto (x, y, z)$ by backpropagation.



PINN — embeds the governing equations in the loss.

Our scope: the plain ANN on the left only. PINNs are used as a *literature benchmark*, not implemented here.

Literature Review

Work	Approach	Gap for our study
Matthews & Bihlo (PinnDE)	PINN + DeepONet on Lorenz-1960	Fixed architecture; physics-informed loss
Aslam et al.	ANN + PSO on a Lorenz variant	Optimises weights, not architecture
Mall & Chakraverty	ANN, 4–6 nodes, sigmoid only	Simple ODEs; no depth/activation sweep
Lau & Werth (ODEN)	Plain ANN; tanh/sigmoid > ReLU	No systematic depth–width study
Dubey / Audu et al.	Plain ANN on first-order ODEs	Fixed nets; not coupled, not Lorenz
Our work	Plain ANN, systematic search	Depth × width × activation on Lorenz-1960

17 papers reviewed across PINNs, pure-ANN solvers, architecture studies, Neural ODEs, and hybrids.

Research Gap & Questions

The gap

- ▶ No architecture study for Lorenz-1960
- ▶ No PINN vs. plain-ANN comparison on it
- ▶ No systematic activation-function study
- ▶ Depth *and* width rarely varied together

Research questions

- ▶ Can a plain ANN approximate the Lorenz-1960 solution from numerical data alone?
- ▶ Which **depth** $\times$ **width** $\times$ **activation** gives the best accuracy–cost trade-off?
- ▶ How close does it get to physics-informed baselines?

Methodology: A Two-Phase Comparative Study

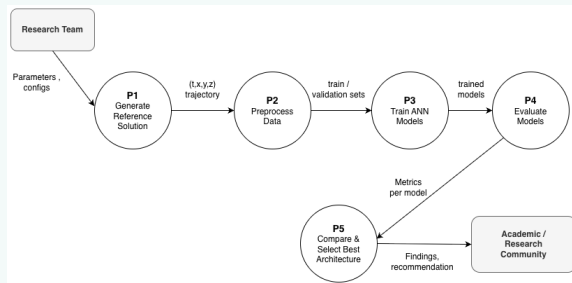
Phase 1 — Architecture search

Depth $\{1, 2, 3, 4\} \times$ width $\{20, 50, 100\} \times$
activation $\{\tanh, \text{ReLU}, \text{sigmoid}, \text{GELU}, \text{Swish}\}$
= 60 runs (optimizer fixed: Adam)

Phase 2 — Optimizers

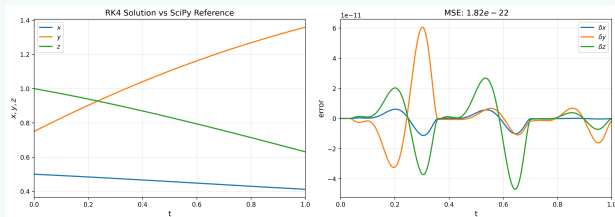
3 best architectures $\times \{\text{Adam}, \text{L-BFGS}, \text{SGD}+\text{momentum}\}$
= 9 runs

Total: 69 controlled experiments



Pipeline: generate → preprocess → train → evaluate →
select.

Progress: Validated Numerical Baseline



RK4 vs. SciPy DOP853 solution (left) and pointwise error (right).

What is done

- Custom RK4 ($h = 10^{-3}$, 1001 points)
- SciPy DOP853 ($\text{rtol} = 10^{-10}$)
- Solvers agree to **MSE**
 $\approx 1.8 \times 10^{-22}$

The reference trajectory is now a **defensible ground truth** for all ANN training in FYDP-II.

Project Plan & FYDP-II Roadmap

Phase	Focus	Status
FYDP-I (wk 1–14)	Problem, literature, requirements, validated baseline	Done
FYDP-II (wk 15–28)	Training pipeline, Phase 1 + Phase 2 runs, metrics & plots	Next
FYDP-III (wk 29–42)	Reproducibility, comparative analysis, final report & defense	Planned

Immediate next steps

- ▶ Build the ANN training/eval pipeline
- ▶ Run the 60 Phase-1 architectures
- ▶ Select the 3 best for Phase-2

Later extensions

- ▶ Repeated seeds for stability
- ▶ Denser grids / new initial conditions
- ▶ Direct comparison with PINN baselines

Conclusion

Where we are

- ▶ Clear, narrow research question
- ▶ 17-paper review with an explicit gap
- ▶ Two-phase design (69 runs) locked
- ▶ Dual-solver baseline verified

Why it matters

- ▶ First systematic plain-ANN architecture study on Lorenz-1960
- ▶ Clean attribution of architecture vs. optimizer effects
- ▶ Honest scope: claims wait for FYDP-II evidence

Foundation set — FYDP-II turns the validated baseline and design into measured results.

Selected References

1. Lorenz, E.N. (1960). Maximum simplification of the dynamic equations. *Tellus*, 12(3).
2. Matthews, J. & Bihlo, A. (2024). PinnDE: physics-informed neural networks for ODEs and PDEs.
3. Raissi, M. et al. (2019). Physics-informed neural networks. *J. Comput. Phys.*, 378.
4. Lagaris, I.E. et al. (1998). ANN for solving ODEs and PDEs. *IEEE Trans. Neural Netw.*, 9(5).
5. Lau, K. & Werth, D. (2020). ODEN: a framework to solve ODEs with neural networks.
6. Mall, S. & Chakraverty, S. (2013). Comparison of ANN architectures for ODEs.
7. Dubey, A. et al. (2024). Solving coupled ODEs using artificial neural networks.
8. Chen, R.T.Q. et al. (2018). Neural ordinary differential equations. *NeurIPS*.

Full bibliography (17 works) in the project report, `paper/final/fydp.bib`.

Thank you

Questions & discussion

Team Lorenz-1960 $\times$ ANN ■ Department of CSE, United International University

Supervisor: Dr. Muhammad Nomani Kabir